

✓ Stakeholder Requirements

Requirement A: The "Delivery Efficiency" Audit

The Problem: We suspect certain "Rider IDs" are consistently slower than others, affecting our NPS (Net Promoter Score).

Task: Create a new column `delivery_duration` (Delivered Time - Order Time). Group by `Rider_ID` and find the top 10 slowest riders who have completed more than 50 orders.

Requirement B: Peak-Hour Surge Analysis

The Problem: Need to know when to trigger "Surge Pricing" to attract more riders.

Task: Extract the Hour from the `Order_Time` column. Identify which 3-hour window (e.g., 6 PM - 9 PM) has the highest volume of orders.

Requirement C: The "Distance vs. Delay" Correlation

The Problem: Marketing wants to know if "Long Distance" is the primary reason for cancellations.

Task: Calculate the correlation between `last_mile_distance` and a boolean `is_cancelled` column.

✓ Importing Libraries

```
# importing all necessary modules
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
import kagglehub as kh
#This module provides a portable way of using operating system dependent functionality
import os
```

```
# importing latest version of dataset from kaggle
path= kh.dataset_download("cbhavik/swiggyzomato-order-information")
print("path to dataset file:", path)
```

```
Downloading to /root/.cache/kagglehub/datasets/cbhavik/swiggyzomato-order-information/2.archive...
100%|██████████| 16.9M/16.9M [00:00<00:00, 109MB/s] Extracting files...
```

```
path to dataset file: /root/.cache/kagglehub/datasets/cbhavik/swiggyzomato-order-information/versions/2
```

```
# getting file using os
files=os.listdir(path)
print(files)
```

```
['Rider-Info.csv']
```

```
# joining path and accessing file using os
df=pd.read_csv(os.path.join(path,"Rider-Info.csv")).copy()
df.head()
```

	order_time	order_id	order_date	allot_time	accept_time	pickup_time	delivered_time	rider_id	first_mile_distance
0	2021-01-26 02:21:35	556753	2021-01-26 00:00:00	2021-01-26 02:21:59	2021-01-26 02:22:08	2021-01-26 02:32:51	2021-01-26 02:49:47	11696	1.5666
1	2021-01-26 02:33:16	556754	2021-01-26 00:00:00	2021-01-26 02:33:57	2021-01-26 02:34:45	2021-01-26 02:50:25	2021-01-26 03:11:15	18117	2.5207
2	2021-01-26 02:39:49	556755	2021-01-26 00:00:00	2021-01-26 02:39:57	2021-01-26 02:40:13	2021-01-26 02:56:00	2021-01-26 03:12:46	18623	2.2074
3	2021-01-26 02:47:53	556756	2021-01-26 00:00:00	2021-01-26 02:48:25	2021-01-26 02:49:06	2021-01-26 03:21:51	2021-01-26 03:41:05	15945	2.1894
4	2021-01-26 03:06:30	556757	2021-01-26 00:00:00	2021-01-26 03:07:21	2021-01-26 03:07:57	2021-01-26 03:31:38	2021-01-26 04:00:15	17589	2.7870

df.tail()

	order_time	order_id	order_date	allot_time	accept_time	pickup_time	delivered_time	rider_id	first_mile_dist
449995	2021-02-06 10:03:16	130226	2021-02-06 00:00:00	2021-02-06 10:03:44	2021-02-06 10:04:14	2021-02-06 10:27:29	2021-02-06 10:44:08	1006	0
449996	2021-02-06 10:03:17	130227	2021-02-06 00:00:00	2021-02-06 10:03:18	2021-02-06 10:04:34	2021-02-06 10:22:17	2021-02-06 10:31:43	279	1
449997	2021-02-06 10:03:18	130228	2021-02-06 00:00:00	2021-02-06 10:04:06	2021-02-06 10:04:39	2021-02-06 10:19:06	2021-02-06 10:26:56	3161	1
449998	2021-02-06 10:03:19	130229	2021-02-06 00:00:00	2021-02-06 10:03:19	2021-02-06 10:05:41	2021-02-06 10:20:39	2021-02-06 10:30:41	9396	2
449999	2021-02-06 10:03:24	130230	2021-02-06 00:00:00	2021-02-06 10:03:45	2021-02-06 10:05:14	2021-02-06 10:13:26	2021-02-06 10:19:41	2078	1

Exploratory Data Analysis-I : MetaData

```
# Rows
df.shape[0]
```

```
450000
```

```
# columns
df.shape[1]
```

```
20
```

```
# shape
df.shape
```

```
(450000, 20)
```

```
# columns name (-first_mile_distance : rider to restaurants, -last_mile_distance : Restaurants to customers)
df.columns
```

```
Index(['order_time', 'order_id', 'order_date', 'allot_time', 'accept_time',
      'pickup_time', 'delivered_time', 'rider_id', 'first_mile_distance',
      'last_mile_distance', 'alloted_orders', 'delivered_orders', 'cancelled',
      'undelivered_orders', 'lifetime_order_count', 'reassignment_method',
      'reassignment_reason', 'reassigned_order', 'session_time',
      'cancelled_time'],
      dtype='object')
```

```
# dataset summary
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 450000 entries, 0 to 449999
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   order_time            450000 non-null object
1   order_id              450000 non-null int64
2   order_date            450000 non-null object
3   allot_time            450000 non-null object
```

4	accept_time	449843	non-null	object
5	pickup_time	447579	non-null	object
6	delivered_time	444782	non-null	object
7	rider_id	450000	non-null	int64
8	first_mile_distance	450000	non-null	float64
9	last_mile_distance	450000	non-null	float64
10	alloted_orders	433052	non-null	float64
11	delivered_orders	432659	non-null	float64
12	cancelled	450000	non-null	int64
13	undelivered_orders	432659	non-null	float64
14	lifetime_order_count	449947	non-null	float64
15	reassignment_method	13744	non-null	object
16	reassignment_reason	13753	non-null	object
17	reassigned_order	13753	non-null	float64
18	session_time	446325	non-null	float64
19	cancelled_time	5218	non-null	object

dtypes: float64(8), int64(3), object(9)

memory usage: 68.7+ MB

```
# statstical summary
df.describe(include='all')
```

	order_time	order_id	order_date	allot_time	accept_time	pickup_time	delivered_time	rider_id	first_mi
count	450000	450000.000000	450000	450000	449843	447579	444782	450000.000000	4
unique	252868	NaN	12	246871	254201	257117	257067	NaN	1
top	2021-01-30 14:38:36	NaN	2021-02-05 00:00:00	2021-01-29 14:18:11	2021-01-27 16:03:12	2021-01-28 15:29:58	2021-01-30 14:38:14	NaN	1
freq	11	NaN	42341	13	15	11	12	NaN	1
mean	NaN	369143.080767	NaN	NaN	NaN	NaN	NaN	7763.244016	1
std	NaN	131146.906408	NaN	NaN	NaN	NaN	NaN	5592.880135	1
min	NaN	118350.000000	NaN	NaN	NaN	NaN	NaN	0.000000	1
25%	NaN	257342.750000	NaN	NaN	NaN	NaN	NaN	2805.000000	1
50%	NaN	369842.500000	NaN	NaN	NaN	NaN	NaN	6754.000000	1
75%	NaN	482342.250000	NaN	NaN	NaN	NaN	NaN	11965.000000	1
max	NaN	594842.000000	NaN	NaN	NaN	NaN	NaN	21566.000000	1

```
# Toal drivers (19537 riders)
df["rider_id"].value_counts()
```

	count
rider_id	
237	228
190	213
11998	209
170	203
853	200
...	...
13174	1
13172	1
7864	1
1069	1
13107	1

19537 rows x 1 columns

dtype: int64

```
df[df["accept_time"].isnull()]
```

	order_time	order_id	order_date	allot_time	accept_time	pickup_time	delivered_time	rider_id	first_mile_dist
1700	2021-01-26 06:35:16	558453	2021-01-26 00:00:00	2021-01-26 06:35:18	NaN	NaN	NaN	15903	1
4066	2021-01-26 07:20:44	560819	2021-01-26 00:00:00	2021-01-26 07:20:45	NaN	NaN	NaN	3430	1
9296	2021-01-26 08:58:12	566050	2021-01-26 00:00:00	2021-01-26 08:58:41	NaN	NaN	NaN	20621	2
13365	2021-01-26 12:42:36	570118	2021-01-26 00:00:00	2021-01-26 12:42:37	NaN	NaN	NaN	10605	0
13429	2021-01-26 12:45:42	570182	2021-01-26 00:00:00	2021-01-26 12:45:43	NaN	NaN	NaN	3585	0
...	...	...	...	...	...	...	...	...	...
436851	2021-02-05 17:43:12	197797	2021-02-05 00:00:00	2021-02-05 17:43:47	NaN	NaN	NaN	14594	0
437757	2021-02-05 17:55:35	198704	2021-02-05 00:00:00	2021-02-05 17:56:36	NaN	NaN	NaN	9156	0
444007	2021-02-06 07:58:56	124238	2021-02-06 00:00:00	2021-02-06 08:03:56	NaN	NaN	NaN	5535	2
445822	2021-02-06 08:30:17	126053	2021-02-06 00:00:00	2021-02-06 08:30:18	NaN	NaN	NaN	178	2
449932	2021-02-06 09:59:40	130163	2021-02-06 00:00:00	2021-02-06 10:04:41	NaN	NaN	NaN	2583	0

157 rows x 20 columns

```
# converting order_time, order_date , allot_time, pickup_time, delivered_time, cancelled_time
df['order_time'] = pd.to_datetime(df['order_time'])
df['order_date'] = pd.to_datetime(df['order_date'])
df['allot_time'] = pd.to_datetime(df['allot_time'])
df['pickup_time'] = pd.to_datetime(df['pickup_time'])
```



```
df['delivered_time'] = pd.to_datetime(df['delivered_time'])
df['cancelled_time'] = pd.to_datetime(df['cancelled_time'])
```

```
# getting desired columns
new_df=pd.concat([df.iloc[:, :15], df.iloc[:, -2:]], axis=1)
```

```
new_df.head()
```

	order_time	order_id	order_date	allot_time	accept_time	pickup_time	delivered_time	rider_id	first_mile_distance
0	2021-01-26 02:21:35	556753	2021-01-26	2021-01-26 02:21:59	2021-01-26 02:22:08	2021-01-26 02:32:51	2021-01-26 02:49:47	11696	1.5666
1	2021-01-26 02:33:16	556754	2021-01-26	2021-01-26 02:33:57	2021-01-26 02:34:45	2021-01-26 02:50:25	2021-01-26 03:11:15	18117	2.5207
2	2021-01-26 02:39:49	556755	2021-01-26	2021-01-26 02:39:57	2021-01-26 02:40:13	2021-01-26 02:56:00	2021-01-26 03:12:46	18623	2.2074
3	2021-01-26 02:47:53	556756	2021-01-26	2021-01-26 02:48:25	2021-01-26 02:49:06	2021-01-26 03:21:51	2021-01-26 03:41:05	15945	2.1894
4	2021-01-26 03:06:30	556757	2021-01-26	2021-01-26 03:07:21	2021-01-26 03:07:57	2021-01-26 03:31:38	2021-01-26 04:00:15	17589	2.7870

```
new_df.shape
```

```
(450000, 17)
```

```
#new dataset into
new_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 450000 entries, 0 to 449999
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   order_time            450000 non-null  datetime64[ns]
```

```

1  order_id          450000 non-null  int64
2  order_date        450000 non-null  datetime64[ns]
3  allot_time        450000 non-null  datetime64[ns]
4  accept_time       449843 non-null  object
5  pickup_time       447579 non-null  datetime64[ns]
6  delivered_time    444782 non-null  datetime64[ns]
7  rider_id          450000 non-null  int64
8  first_mile_distance 450000 non-null  float64
9  last_mile_distance 450000 non-null  float64
10 allotted_orders   433052 non-null  float64
11 delivered_orders   432659 non-null  float64
12 cancelled          450000 non-null  int64
13 undelivered_orders 432659 non-null  float64
14 lifetime_order_count 449947 non-null  float64
15 session_time       446325 non-null  float64
16 cancelled_time     5218 non-null  datetime64[ns]
dtypes: datetime64[ns](6), float64(7), int64(3), object(1)
memory usage: 58.4+ MB

```

```

# getting order day, week, weekdays, month, year
new_df['order_year']= new_df['order_date'].dt.year
new_df['order_month']= new_df['order_date'].dt.month
new_df['order_month_name'] = new_df['order_date'].dt.month_name()
new_df['order_day']= new_df['order_date'].dt.day
new_df['order_week']= new_df['order_date'].dt.isocalendar().week
new_df['order_hour']= new_df['order_time'].dt.hour # we are getting hours in 24 hrs format
new_df['order_day_name'] = new_df['order_date'].dt.day_name()
# time difference between ordered item to delivered time in hours
# Correcting the order to get a positive duration and converting to hours
new_df['delivery_duration'] = (new_df['delivered_time'] - new_df['order_time']).dt.total_seconds() / 3600
# pickup time took by rider to pick up delivery in hours
# Correcting the order to get a positive duration and converting to hours
new_df['pick_up_duraton_overall'] = (new_df['pickup_time'] - new_df['order_time']).dt.total_seconds() / 3600

```

```

# date range
start_date=new_df['order_date'].min()
end_date = new_df['order_date'].max()
print(start_date, end_date)

```

2021-01-26 00:00:00 2021-02-06 00:00:00

✓ Univariate Analysis

`new_df.head()`

	order_time	order_id	order_date	allot_time	accept_time	pickup_time	delivered_time	rider_id	first_mile_distance
0	2021-01-26 02:21:35	556753	2021-01-26	2021-01-26 02:21:59	2021-01-26 02:22:08	2021-01-26 02:32:51	2021-01-26 02:49:47	11696	1.5666
1	2021-01-26 02:33:16	556754	2021-01-26	2021-01-26 02:33:57	2021-01-26 02:34:45	2021-01-26 02:50:25	2021-01-26 03:11:15	18117	2.5207
2	2021-01-26 02:39:49	556755	2021-01-26	2021-01-26 02:39:57	2021-01-26 02:40:13	2021-01-26 02:56:00	2021-01-26 03:12:46	18623	2.2074
3	2021-01-26 02:47:53	556756	2021-01-26	2021-01-26 02:48:25	2021-01-26 02:49:06	2021-01-26 03:21:51	2021-01-26 03:41:05	15945	2.1894
4	2021-01-26 03:06:30	556757	2021-01-26	2021-01-26 03:07:21	2021-01-26 03:07:57	2021-01-26 03:31:38	2021-01-26 04:00:15	17589	2.7870

5 rows x 26 columns

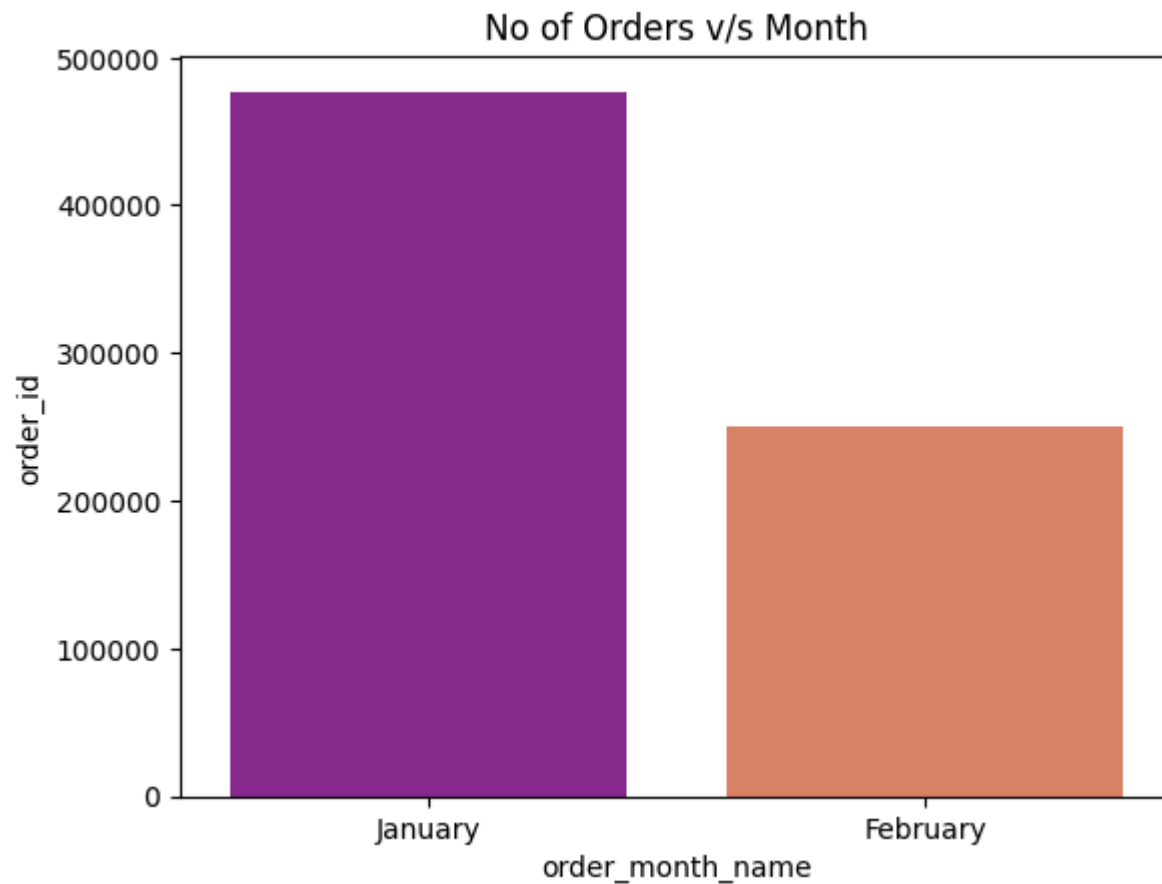
✓ Order v/s State

```
# orders v/s month
sns.barplot(x="order_month_name", y="order_id", data=new_df, palette="plasma")
plt.title("No of Orders v/s Month")
```

```
/tmp/ipykernel_2207/1628118137.py:2: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hu
```

```
sns.barplot(x="order_month_name", y="order_id", data=new_df, palette="plasma")  
Text(0.5, 1.0, 'No of Orders v/s Month')
```

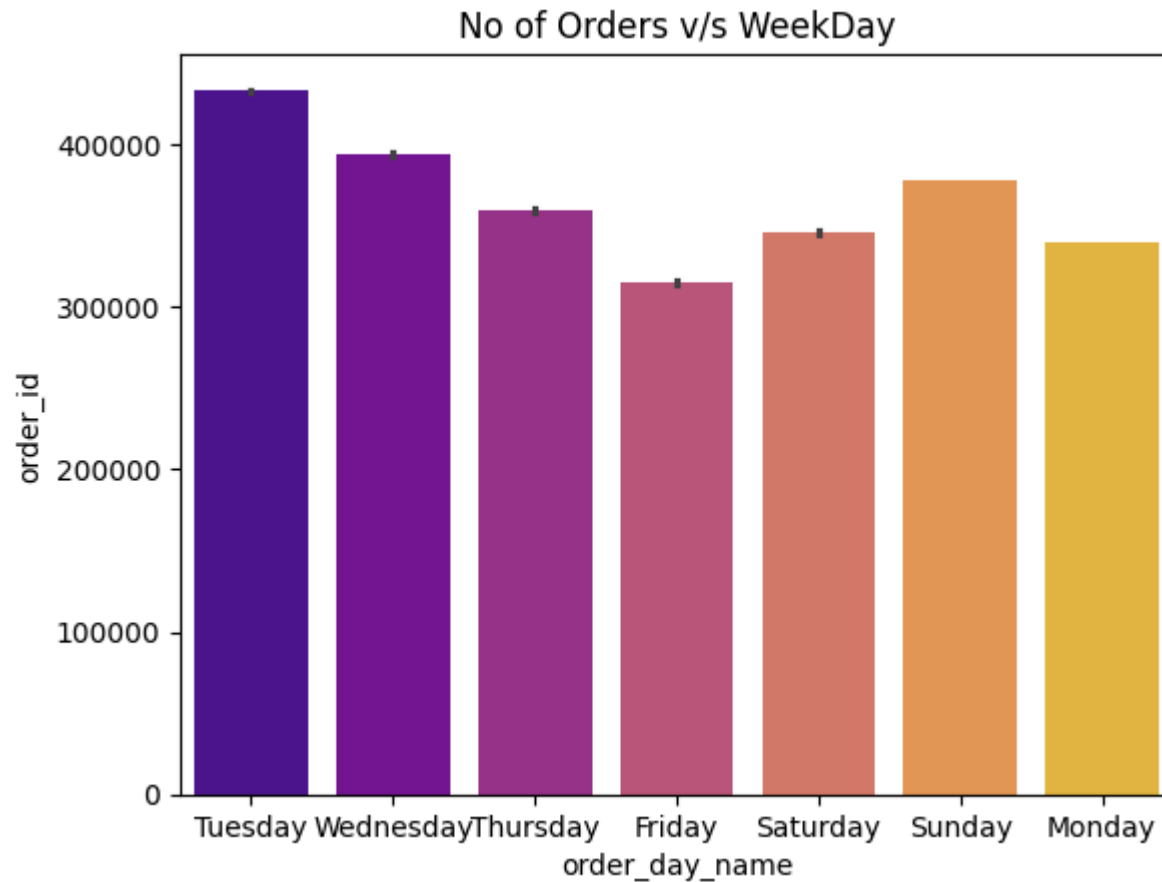


```
# Orders v/s WeekDay  
sns.barplot(x="order_day_name", y="order_id", data= new_df, palette="plasma")  
plt.title("No of Orders v/s WeekDay")
```

```
/tmp/ipykernel_2207/3910961807.py:2: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hu
```

```
sns.barplot(x="order_day_name", y="order_id", data= new_df, palette="plasma")  
Text(0.5, 1.0, 'No of Orders v/s WeekDay')
```

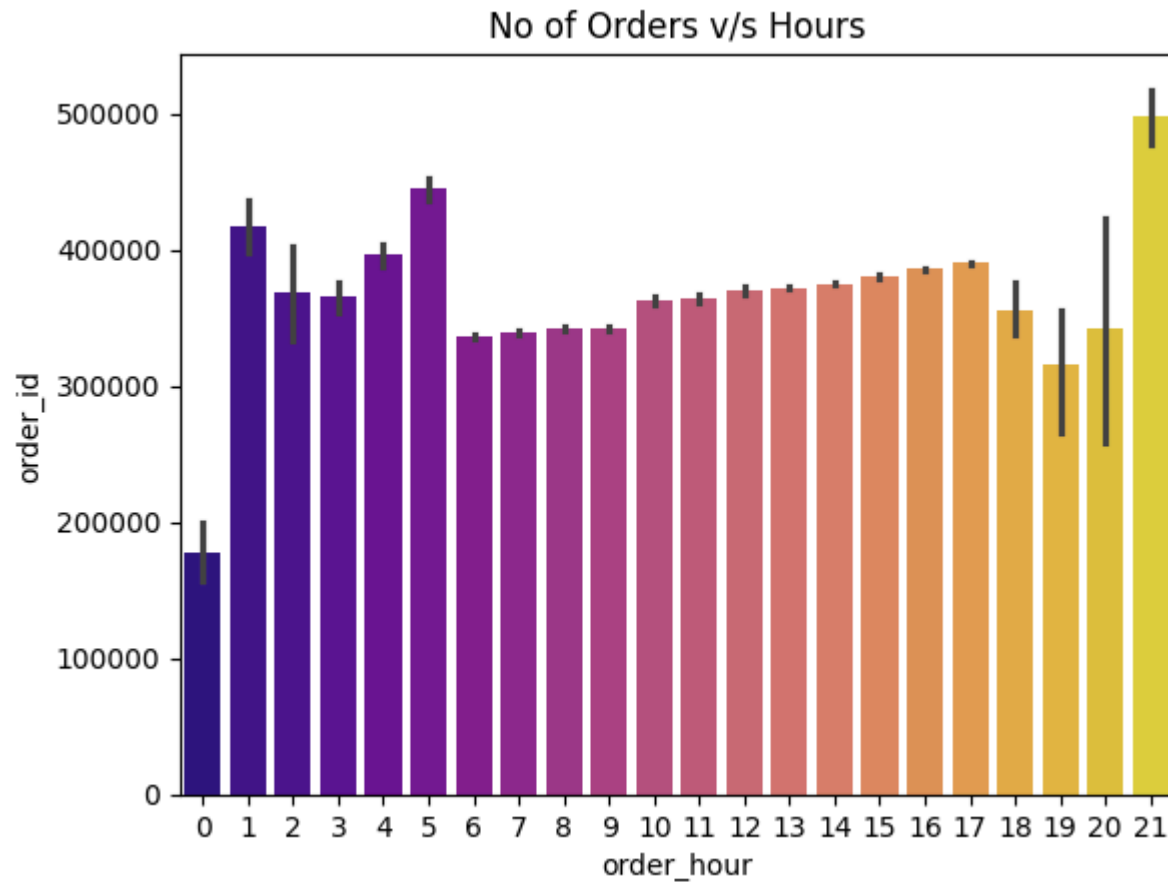


```
# orders v/s hours  
sns.barplot(x=new_df['order_hour'], y=new_df['order_id'], palette='plasma')  
plt.title("No of Orders v/s Hours")
```

```
/tmp/ipykernel_2207/1883841376.py:2: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hu
```

```
sns.barplot(x=new_df['order_hour'], y=new_df['order_id'], palette='plasma')  
Text(0.5, 1.0, 'No of Orders v/s Hours')
```



```
# converting different time in different buckets
```

```
bin = [0,3,6,9,12,15,18,21,24]
```

```
label = ["Late Night", "Early Morning", "Morning", "Noon", "Afternoon", "Eve", "Early_Night", "Night"]
```

```
new_df['hours_bucket'] = pd.cut(new_df['order_hour'], bins=bin, labels=label, include_lowest=True)
```

```
# checking which bucket get which label  
new_df[['order_hour', 'hours_bucket']].drop_duplicates().sort_values(by='order_hour')
```

order_hour	hours_bucket	
356093	0	Late Night
117888	1	Late Night
0	2	Late Night
4	3	Late Night
124	4	Early Morning
424	5	Early Morning
877	6	Early Morning
3010	7	Morning
6214	8	Morning
9386	9	Morning
11251	10	Noon
12047	11	Noon
12766	12	Noon
13638	13	Afternoon
17340	14	Afternoon
22962	15	Afternoon
28532	16	Eve
33854	17	Eve
38087	18	Eve
77104	19	Early_Night
117886	20	Early_Night
117887	21	Early_Night




```
##Requirement B: Peak-Hour Surge Analysis
sns.barplot(x='hours_bucket', y='order_id', data=new_df, palette='plasma')
plt.title('No of Orders v/s Hours Bucket')
```

/tmp/ipykernel_2207/2683709530.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` to silence this warning

```
sns.barplot(x='hours_bucket', y='order_id', data=new_df, palette='plasma')
Text(0.5, 1.0, 'No of Orders v/s Hours Bucket')
```

